Arduino C. To illustrate this point, Listings 1-1 and 1-2 provide two examples of the same source code to blink the onboard LED connected to digital pin 13.

Listing 1-1. *Blink LED with avr-libc*

```
#include <avr/io.h>
#include <util/delay.h>

int main(void) {
  while (1) {
    PORTB = 0x20;
    _delay_ms(1000);
    PORTB = 0x00;
    _delay_ms(1000);
  }
  return 1;
}
```

Listing 1-2. *Blink LED with Arduino*

```
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

These two different listings show two functionally identical sketches, one written with the Arduino library and one written without. The really nifty thing here is that, if you want to geek out, the Arduino development environment is fully compatible and extensible using C/C++ code written using the avr-libc library, a subset of the standard C library, and the GCC compiler, both written for Atmel's standard 8-bit AVR microcontrollers. Listing 1-1 is written with avr-libc while Listing 1-2 is written using the Arduino library. They both are compatible with the Arduino development environment and can be uploaded the same way to the Arduino board. The first example also consumes a fifth of the amount of memory as the second example, coming in at 210 bytes as opposed to 1010 bytes.

What the Arduino example lacks in memory efficiency, however, it more than makes up for in usability and integration with the Arduino interface board. For example, referring to the digital pin that our LED is connected to as pin number 13 is generally easier for most people than the hexadecimal address 0x20 on PORTB. This simplicity is one the benefits to writing code using Arduino C. That is not to say that one is better than the other, but simply that this scalability and flexibility is an often-overlooked benefit of learning on the Arduino platform because it allows budding code-monkeys the opportunity to develop into ever more powerful architectures later. We will focus on programming the Arduino using the standard Arduino libraries, although if you want to know more, full documentation on the avr-libc library package can be found at `www.nongnu.org/avr-libc/`. While we are at it, it is also worth mentioning that it is even possible to program the Arduino interface board using other development environments more often associated with computer development, such as Eclipse, NetBeans, or any other development package that you are familiar with … or if you have an aversion to the color teal.